AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

RAHUL RAJ

STUDENT ID: 12341680, Indian Institute of Technology Bhilai, India

Village ponds harvest monsoon runoff, but siting one needs terrain, drainage, rainfall and storage analysis that a Gram Panchayat rarely has. We built a web application in which an administrator either draws a box around the village land on a satellite map or uploads a KML/KMZ contour map, and receives a ranked pond location, the catchment draining to it, the expected water volume and a costed pond design, all overlaid on the map. Elevation comes from the upload (Delaunay TIN) or from the Copernicus GLO-30 DEM for the drawn box; both feed one validated hydrology chain: Priority-Flood conditioning, D8 flow routing as a forest of in-trees, a single topological sweep for accumulation and for a complete-catchment test, multi-criteria site scoring with hard exclusions for rivers and mapped water, and catchment delineation by reverse breadth-first search. Runoff is SCS-CN on 45 years of daily ERA5-Land rainfall; the pond is sized by cost and checked by a daily water balance. Every number carries a unit and an uncertainty band. The catchment engine matches the independent pysheds implementation within 2–3 % on the same DEM; 220 tests (41 golden) pass; the end-to-end HTTP suite passes on four lab replicas behind a load balancer. On one process a village-sized selection is analysed in 2 s (median) under 20 concurrent users.

Additional Key Words and Phrases: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, D8 Flow Routing, Web GIS, REST APIs

ACM Reference Format:

Rahul Raj, Student ID: 12341680. 2026. AI-based Village Pond Planning System: CSD Assignment 1 – Final Technical Report. 1, 1 (October 2026), 14 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Rural India stores monsoon water in small excavated ponds and tanks. A pond is only useful where the terrain delivers enough runoff to fill it and not so much that the embankment becomes a river dam. This report describes a system that finds such places from public geospatial data and explains its answer. Section 2 maps the requirements to modules; Sections 3–4 give the high- and low-level design and the algorithms (the examined core); Sections 5–6 cover implementation and the CSD themes; Section 7 evaluates; Section 8 states the limits.

1.1 Motivation

Siting by hand needs a survey, a rainfall record and runoff and storage calculations; contour sheets are rarely digitised, gauges are sparse and land records are not public. Yet the questions are computable — where does water converge,

Author's address: Rahul Raj
Student ID: 12341680, Indian Institute of Technology Bhilai, Bhilai, India, rahul.raj9237@gmail.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Association for Computing Machinery.

Manuscript submitted to CSD Assignment

how much land drains there, how much rain falls in a 75 %-dependable year, what pond does that fill — so a tool that answers them in seconds, with stated uncertainty, lets an administrator shortlist sites before paying for a survey.

1.2 Scope of the Project

In scope: selecting a land area on a map (a drawn box) or uploading a KML/KMZ contour map; terrain products (DEM, slope, TWI, contours, streams); ranked pond sites with per-criterion scores; catchment of any clicked point; 45-year rainfall statistics; runoff by three methods; pond depth, dimensions, storage, fill reliability, spillway and indicative cost; land-availability and water masks; saving, approving and exporting a recommendation. **Out of scope:** structural design of the embankment, groundwater recharge modelling, legal land ownership (never assumed; shown as unknown), and survey-grade accuracy — all results are planning grade from 30 m public DEMs.

2 PROBLEM STATEMENT AND REQUIREMENTS

The assignment asks for a web application that analyses terrain and rainfall to recommend pond sites and estimates catchment, runoff, depth and storage on an interactive map. The Phase 3 brief adds: select the land area on a map; return the suggested pond location, catchment area and expected water volume for it, overlaid on the map; stay fast within the four provided lab systems. Table 1 maps each requirement to its module.

Table 1. Functional requirements and where they are implemented

Requirement	Implemented in
Select land area on a map (Phase 3)	`web/src/components/AreaPanel.tsx`, `MapView.tsx`; `POST /analyzeArea`; `engines/terrain/adapters.py` (`ProviderTileAdapter`)
Satellite imagery display	Esri World Imagery basemap in `MapView.tsx`; village named by reverse geocoding (`providers/geocoding.py`)
Contour map visualization	`engines/terrain/contours.py` (marching squares + Douglas–Peucker); `GET /terrain/{id}/contours`
Available-land identification	`engines/suitability/constraints.py` (Specification pattern), `water_mask.py` (NDWI + OpenCV)
Catchment area estimation	`engines/hydrology/{conditioning,flow,catchment}.py`; `POST /analysis/catchment`
Suggested pond location	`engines/hydrology/siting.py` (MCDA + hard exclusions + NMS)
Historical rainfall query	`providers/rainfall/` (Open-Meteo ERA5-Land, NASA POWER), `engines/rainfall/statistics.py`
Runoff volume estimation	`engines/runoff/` (SCS-CN on the daily series, rational, Strange)
Pond depth / storage recommendation	`engines/design/` (`PondDesignBuilder`, EAV, water balance, spillway)
Combined overlay / results view	`web/src/components/ResultsOverlay.tsx` + map layers

2.1 Non-Functional Requirements

Performance: a village-sized area (0.25–25 km^2) analysed end to end in under 10 s; a catchment click under 2 s. **Scalability and stress:** long work runs as asynchronous jobs on bounded pools (bulkheads); a full queue answers 429 rather than queueing without bound; request size is capped by area; replicas scale horizontally behind a load balancer. **Availability:** external providers are wrapped in retry, circuit breaker, cache and fallback; a dead replica is removed from rotation; the frontend works offline from cache. **Correctness and honesty:** every number carries a unit and an uncertainty band; warnings name every assumption. **Security:** JWT (RS256) with viewer/planner/officer roles on the

approval workflow; append-only audit log. **Usability:** works on current Chrome/Firefox/Safari and at 390 px width; Lighthouse accessibility 100.

3 SYSTEM ARCHITECTURE AND HIGH-LEVEL DESIGN

The system is a layered monolith (ADR 0001) behind asynchronous job routes (Figure 1). The browser talks to one origin. The API validates and enqueues; a worker runs the use-case workflow, which sequences pure engines and calls external data only through adapters. Dependencies point inward only — `api` → `engines` → `providers`/`repositories` → `domain` — and a test (`tests/test_layering.py`) fails the build if a router holds business logic or an engine imports a framework.

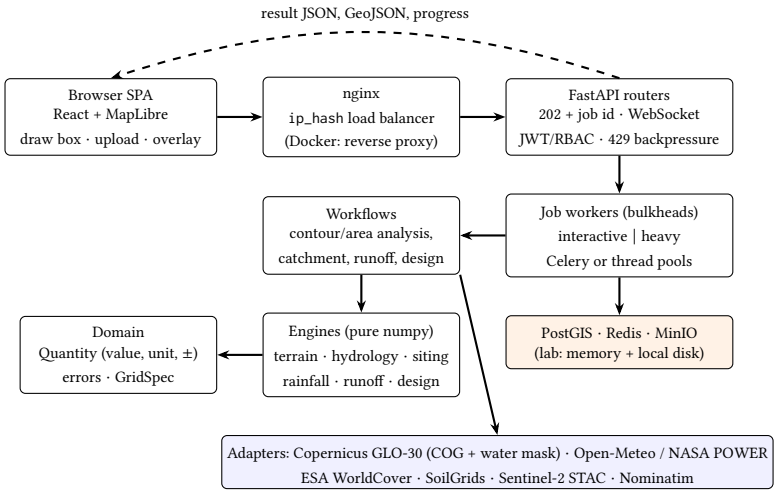


Fig. 1. High-level system architecture. Arrows show the request path of an analysis: the UI posts a box or a file, receives `202` with a job id, and follows progress over a WebSocket (polling fallback) until the result is overlaid on the map.

Work flow of one analysis (Figure 2): the request is admitted in milliseconds, the heavy work runs on a worker, and the browser follows progress until it can draw the result and request the pond design.

Two deployment shapes, one code base: an 11-service Docker Compose stack (PostGIS, Redis, MinIO, API, two Celery workers, beat, TiTiler, nginx, Prometheus, Grafana), and on the four lab machines — unprivileged containers without Docker or root — one process per VM (in-memory persistence, thread-pool jobs, local store) behind a user-space nginx load balancer. The switch is configuration only (ports and adapters, ADR 0013).

3.1 Technology Stack

Python 3.12, FastAPI, Pydantic v2; numpy/scipy for every algorithm; rasterio/GDAL and pyproj for rasters and UTM; OpenCV for the water mask; PostgreSQL + PostGIS, Redis + Celery, MinIO; React + TypeScript + MapLibre GL; Docker Compose; GitHub Actions. Elevation: the uploaded contour map, or Copernicus GLO-30 COGs [5] for a drawn box.

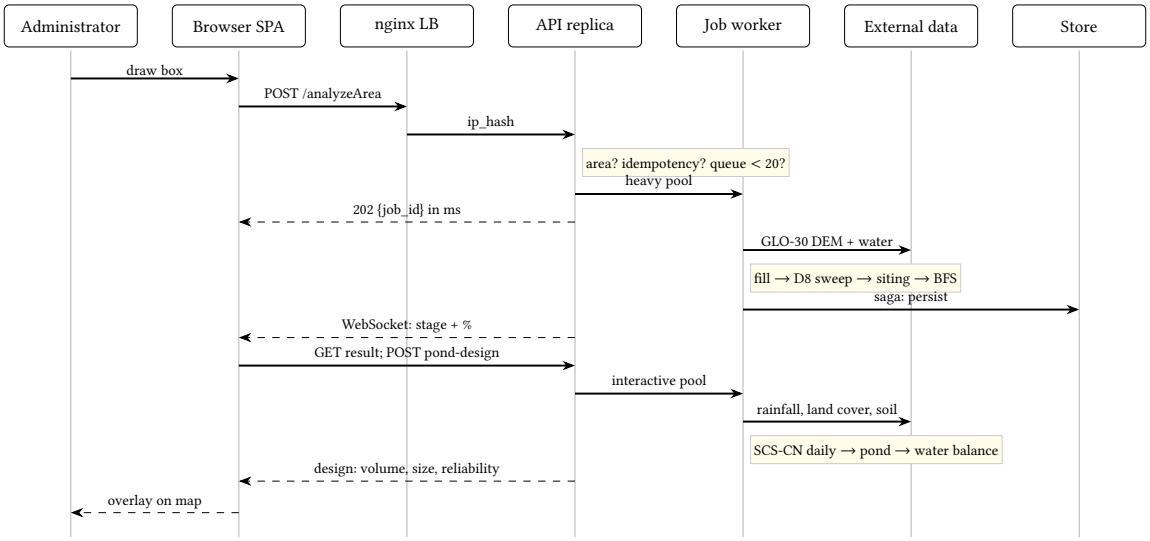


Fig. 2. Work flow of one map-selected analysis (sequence diagram). Solid arrows are requests, dashed are responses; the two jobs run on different bulkheads.

Rainfall: Open-Meteo ERA5-Land [9] with NASA POWER as fallback. **Deviations from the suggested stack, justified:** FastAPI over Flask (typed OpenAPI contract generated from the code, dependency injection for ports); PostgreSQL + PostGIS over MongoDB (spatial types, boundary reuse by `ST_Equals`, append-only audit rules in the database); full DEM rasters over point-elevation APIs (flow direction needs a grid — a village is ∼10 000 cells, one COG window read versus thousands of throttled point calls); ERA5-Land over IMD (keyless, daily, 45 years in 2 s; IMD gridded data needs registration and manual download).

3.2 Low-Level Design

- **Ports and adapters.** `DEMProvider` has two adapters: `ContourKMLAdapter` (parse, infer provenance, TIN) and `ProviderTileAdapter` (GLO-30 window read and warp). The analysis workflow's *only* branch on input kind is which adapter to build. `JobRunner` has three (Celery, thread pools, inline for tests). Each rainfall provider is wrapped as `Cached(CircuitBreaker(Retry(p)))` inside a `FallbackChain`.
- **Contract.** Every analysis route returns `JobAccepted{job_id, poll_url}`. Result schemas (`ContourAnalysisResult`, `CatchmentResult`, `PondDesignResult`) are Pydantic models from which `openapi.json` and the frontend's TypeScript types are generated. Every numeric field is a `Quantity` {value, unit, uncertainty_pct, low, high, method} — a domain type, so a bare float cannot reach the API.
- **Persistence** of an analysis is a Saga of four idempotent steps with compensations run in reverse on failure. **Data model:** `villages` (PostGIS boundary), `dem_assets` (provenance, statistics, object keys), `jobs` (status, stage, progress, result JSON, idempotency key), `recommendations` + outbox + append-only `audit_log`; rasters are Cloud-Optimised GeoTIFFs in the object store.

4 METHODOLOGY

Figure 3 is the analysis as a flowchart: two ways in, one chain, and every place where the system refuses, relaxes or warns instead of guessing. The subsections give each stage.

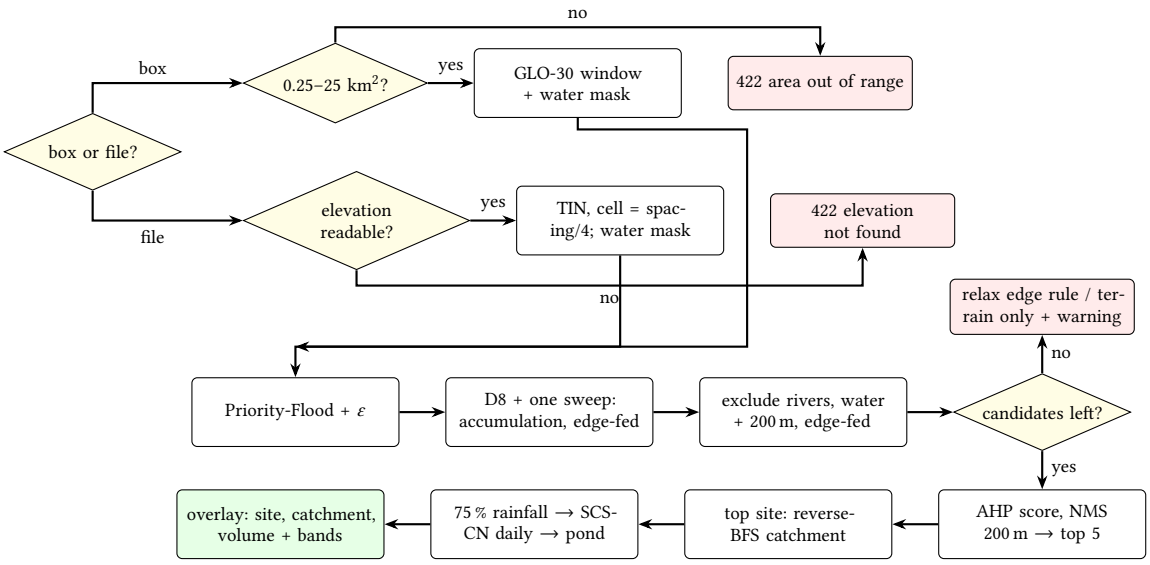


Fig. 3. Analysis flowchart. Red boxes are the refusals and fallbacks that make the system say "cannot" rather than guess; every other step is a named algorithm in this section.

4.1 Terrain and Elevation Analysis

Map-selected area. The box's centroid fixes the UTM zone; the grid is the envelope of the projected box at GLO-30's native 30 m. For every 1° tile the grid touches, the window under the grid's own lon/lat footprint (plus two source cells) is read from the COG over HTTP — a few hundred kB — and warped bilinearly onto the grid; tiles are mosaicked. The tile's Water Body Mask (lake/river classes) is warped by nearest neighbour onto the same grid. Limits of 0.25–25 km^2, measured in metres, return `422 area_out_of_range` outside them. **Uploaded contour map.** The KML/KMZ is parsed namespace-agnostically; elevation is read by an ordered strategy (Z → whitelisted `ExtendedData` field → placemark name), rejecting numeric `ID` decoys — the provided sample has one. Vertices are densified and triangulated (Delaunay TIN, linear interpolation, Gaussian σ=1 cell). The cell size is derived, not configured: mean contour spacing (hull area / total contour length) / 4, floored at the source DEM's resolution, which is inferred from the file's own metadata (the sample: SRTM, 30 m). **Surfaces.** Slope, aspect and hillshade by Horn's 3×3 kernel [7]; profile and plan curvature [19]; topographic wetness index $\ln(a/\tan\beta)$ [3]; contours by marching squares, simplified by Douglas–Peucker [4]. **OpenCV** builds the existing-water mask from Sentinel-2 NDWI [8]: Otsu threshold [11], morphological open/close, connected components with an area filter.

4.2 Catchment Area Delineation

Conditioning. Priority-Flood with ε [1, 17] fills every depression to its spill level and gives flats a tiny gradient in one $O(n \log n)$ pass, so every cell drains. **The flow graph.** D8 [10] gives each cell one out-edge to its steepest-descending

neighbour (diagonal drops divided by $\sqrt{2}$). Every node has out-degree ≤ 1 and the filled surface has no cycles, so the graph is a forest of in-trees rooted at map-edge outlets, and *descending filled elevation is a topological order*. One sorted sweep therefore computes flow accumulation (subtree size) and, with OR in place of +, whether a cell's catchment reaches the map edge (Algorithm 1). Streams are cells draining ≥ 5 ha (an area, so it means the same at any resolution), ordered by Strahler [14]. **Delineation.** A click is snapped to the *nearest* cell within 150 m that drains ≥ 2 ha — nearest, not largest, so a tributary click is not dragged onto the river — and the distance moved is shown. The catchment is the reverse breadth-first search from the outlet over the inverted D8 edges (Algorithm 2); area, longest flow path (deepest BFS level) and relief come from the same walk. Area uncertainty is max(15%, one cell ring around the perimeter). *Assumptions and limits:* D8 cannot split flow (D-∞ is a documented stub); on very flat land a 2 m DEM error can redirect a small flow path (Section 7); the result is bounded by the selected area — hence the complete-catchment constraint below.

Algorithm 1 One topological sweep: accumulation and edge-fed flags — $O(n \log n)$

Require: filled DEM z, receiver $r(c)$ from D8 (-1 at outlets)
$acc(c) \leftarrow 1$; $edge(c) \leftarrow$ [c on the grid border or beside nodata] **for all** c
for c **in** cells sorted by z descending **do** ▷ a topological order of the D8 forest
  if $r(c) \neq -1$ **then**
    $acc(r(c)) \leftarrow acc(r(c)) + acc(c)$ ▷ subtree size = upstream cells
    $edge(r(c)) \leftarrow edge(r(c)) \vee edge(c)$ ▷ catchment reaches the map edge
return $acc\times$ cell area, $edge$

Algorithm 2 Catchment of a pour point — reverse BFS, O(catchment size)

Require: outlet o after snapping; donor lists $D(c) = \{d : r(d) = c\}$ (inverse D8, built once)
$Q \leftarrow [o]$; $seen \leftarrow \{o\}$; $level(o) \leftarrow 0$
while Q not empty **do**
  $c \leftarrow Q$.POP()
  for $d \in D(c)$ **do** $seen \leftarrow seen \cup \{d\}$; $level(d) \leftarrow level(c) + \ell(d, c)$; Q.PUSH(d)
return mask $seen$, area $|seen|\cdot$cell area, longest flow path max $level$

4.3 Site Selection — How the Area Is Chosen

The suggested location is the best-scoring cell of the drainage network after *hard exclusions* (Algorithm 3). Scoring is a weighted sum of four memberships in $[0, 1]$ with weights from a Saaty pairwise matrix (principal eigenvector, consistency ratio 0.004 [13]): *upstream area* as a plateau in $\log_{10}$ ha (0 at 1 ha, 1 from 10 to 150 ha, 0 at 1 000 ha — enough to fill a pond, not a river), *flatness* (1 on 0–3 % slope, 0 at 15 %), *wetness* (normalised TWI) and *impoundment efficiency* (volume held behind a nominal 2 m rise by an 8-connected flood fill on the upstream side, divided by its footprint — natural basins score high). Exclusions come first because a score cannot express "never": channels beyond 1 000 ha and mapped water are rivers; nothing within a 200 m flood belt of a channel beyond 150 ha or of mapped water; and nothing whose catchment reaches the map edge, because its area and runoff would be lower bounds. Non-maximum suppression at 200 m keeps the top five distinct.

4.4 Rainfall Data Integration

Daily precipitation 1981–2025 at the site from Open-Meteo ERA5-Land, NASA POWER as fallback (retry, circuit breaker, single-flight cache). Complete calendar years only: mean, CV, **75 % dependable rainfall by the Weibull**

Algorithm 3 Pond site selection (terrain MCDA with hard exclusions)

Require: streams S, upstream area A, slope s, TWI, water mask W, edge-fed flags E

$river \leftarrow (S \wedge A \geq 1000\,\text{ha}) \vee W$; $\quad major \leftarrow (S \wedge A \geq 150\,\text{ha}) \vee W$

$C \leftarrow S \wedge (s \leq 15\%) \wedge \neg river \wedge \textsc{dist}(major) \geq 200\,\text{m}$ ▷ distance transform

if $C \wedge \neg E \neq \emptyset$ **then** $C \leftarrow C \wedge \neg E$ **else** warn `no_complete_catchment`

for $c \in C$ **do** $score(c) \leftarrow \sum_k w_k\, \mu_k(c)$ ▷ area plateau, flatness, TWI, impoundment

return top 5 by $score$ with non-maximum suppression (200 m), each with its μ_k

plotting position $P = m/(n+1)$ (the design year of Indian minor-irrigation practice), June–September share, IMD rainy days (≥ 2.5 mm), and the 25-year 1-day depth by Gumbel EV1 [6].

4.5 Runoff and Pond Sizing

Land cover from ESA WorldCover [18] and soil texture from SoilGrids [12] give an area-weighted TR-55 curve number [16]. SCS-CN $Q = (P - 0.2S)^2/(P + 0.8S)$, $S = 25400/CN - 254$ mm, is applied to *each day* and summed — applied to annual totals it overestimates more than threefold (asserted by a test). The rational coefficient and Strange's table [15] bracket it; the spread is reported. **Expected water volume** = 75 % dependable annual SCS-CN runoff × catchment area. Design: target = runoff × 0.6 harvest efficiency, clamped to 2 000–50 000 m^3; an excavated frustum (2:1 sides, 0.5 m freeboard) whose depth (1.5–3.5 m) and aspect are chosen by minimum cost; a daily water balance (inflow, evaporation, 2 mm/day seepage) gives the share of years the pond fills; the spillway passes the 25-year storm.

5 IMPLEMENTATION

5.1 Backend and API Design

REST under `/api/v1`, OpenAPI 3.1 generated from the code (Swagger at `/docs`). Key routes:

`POST /analyzeArea`	box → 202 job: site, catchment, terrain from GLO-30
`POST /analyzeContour`	KML/KMZ (`contour_map`) → 202 job, same result shape
`GET /jobs/{id}` · `WS /jobs/{id}/ws`	status, stage, percent
`GET /analysis/results/contour/{job}`	suggested location, rationale, catchment, 5 ranked sites, method
`POST /analysis/catchment`	catchment of any point (snap, BFS, metrics)
`GET /rainfall/statistics`	45-year statistics at a point
`POST /analysis/runoff` · `/pond-design`	runoff range; full design with reliability and cost
`POST /analysis/suitability`	land constraints, NDWI water mask, AHP ranking
`GET /terrain/{id}/{contours,streams,layers}`	map layers
`/recommendations`, `/auth/token`	draft → submitted → approved (RBAC), audit, PDF/GeoJSON/CSV

Errors are RFC 9457 problem documents with stable codes generated from the same table the handlers use (`GET /meta/errors`). Third-party failures degrade, never crash: rainfall falls back provider → cache → recorded record; land cover and soil fall back to stated defaults with a warning and a lower confidence label; an unreachable water mask or geocoder is skipped and said so. Authentication: RS256 JWT, roles viewer, planner, officer; approvals require officer.

5.2 Frontend and Visualization

A React + MapLibre single-page app (Figure 4). *Select land area*: search a place, click two opposite corners; the box's area is checked live against the limits. *Analyse* posts the box, shows the job's stage and percentage, then draws the

selected area, contours with labels, streams by Strahler order, the five ranked sites, the top site's catchment polygon and outlet, and — after the pond design runs automatically on the top site — the pond footprint and a results overlay with location, catchment, 75 % rainfall, SCS-CN runoff, storage, dimensions, fill reliability and a confidence badge. Any map click delineates that point's catchment. Nineteen layers are toggleable; an offline-first service worker serves cached results under an "Offline" badge.

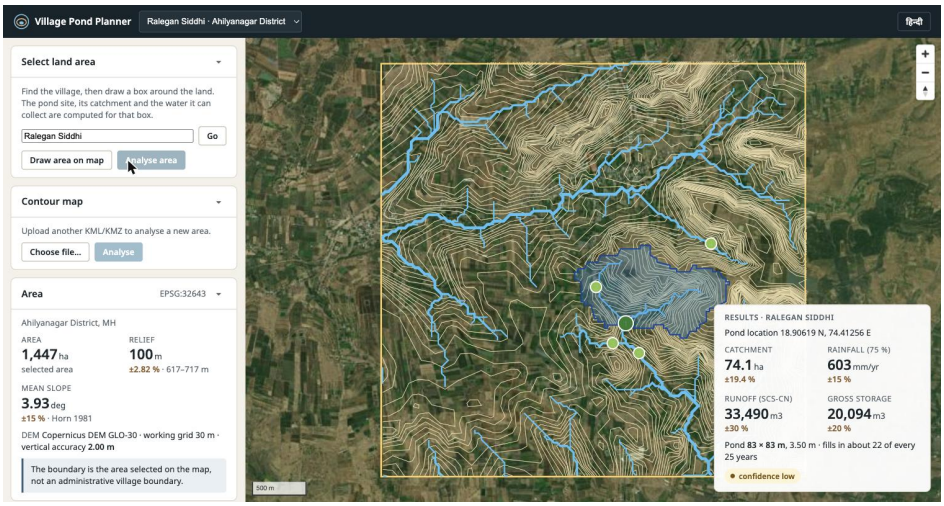


Fig. 4. Application interface: a 14.5 km^2 box drawn at Ralegan Siddhi (Maharashtra), analysed from Copernicus GLO-30 — contours, streams, five ranked sites, the top site's 74 ha catchment and the results overlay (runoff 33 490 m^3, storage 20 094 m^3).

5.3 Database and Storage

PostGIS holds villages (an identical boundary reuses the village), DEM assets, jobs and recommendations; state transitions write an outbox drained into an append-only `audit_log`. Rasters are COGs in MinIO served as tiles by TiTiler; streams, siting and cached rainfall are JSON beside them. On the lab VMs the same ports use memory and local disk.

6 CSD THEMES AND TOPICS APPLIED IN THE PROJECT

Load balancing without shared state. Lab replicas keep villages and jobs in memory (no PostgreSQL/Redis on the VMs), so nginx `ip_hash` keeps a user on one replica, `max_fails` drops a dead one and `proxy_next_upstream` retries elsewhere; the stated cost is a 2 s re-analysis after failover. **Bulkheads over more threads.** The first lab load test showed `POST /analyzeArea` blocking 11–21 s (the job ran inside the request); separate *interactive* and *heavy* thread pools return `202` in milliseconds, and the heavy pool stays small because the hydrology loops are GIL-bound — throughput comes from replicas. **Single-flight caching** cut a cold-herd rainfall p95 from 43 s to 63 ms: concurrent misses wait for one upstream call.

7 RESULTS AND EVALUATION

Provided contour map (Khapri, Durg, Chhattisgarh; 1 355 contours, 267–298 m, SRTM 30 m source; UTM 44N derived from the file). The river entering from outside the map (the Shivnath) carries no in-map accumulation, so an early

Table 2. Mapping of core CSD themes/topics to their use in this project

CSD theme	Where used	Justification
API design (REST)	`/api/v1`, 202 + job id, problem details, generated OpenAPI	long work must not hold a connection; clients branch on stable codes
Load balancing	nginx `ip_hash` over four lab replicas, passive health checks	in-memory replicas need client affinity; a dead VM leaves rotation
Caching	single-flight rainfall cache; LRU GLO-30 windows; COG tiles; service worker	45-year series once per point; a cold herd costs one upstream call
Indexing / query optimisation	PostGIS geometry, unique idempotency key, COG internal tiling	village reuse by boundary; byte-range reads of only the needed window
Concurrency / async	bulkhead pools (interactive, heavy), WebSocket progress, 429 backpressure	a village analysis cannot delay a click; bounded queues under stress
Microservices vs. monolith	layered monolith + workers	one team, one deployable; boundaries enforced by a test, not a network
Design patterns	Ports & Adapters, Strategy, Decorator, Chain of Responsibility, Builder, Specification, Saga, State, Outbox, Observer	each solves a named problem (Section 3)
AuthN / AuthZ	RS256 JWT, three roles, officer-only approval	approvals are accountable; audit log is append-only
Containerisation / deployment	Docker Compose (11 services); one-process replicas on the lab VMs	same code, two shapes, chosen by configuration
Error handling / resilience	retry, circuit breaker, fallback chain, stale cache, stated defaults	degrade with a warning rather than fail
Algorithms and complexity	Priority-Flood $O(n \log n)$, topological sweep $O(n)$, reverse BFS, distance transform	exact, textbook, explainable; ~2 s per village
Testing strategy	220 tests: 41 golden (analytic terrain), API flows, pysheds cross-check, e2e over HTTP (Section 7.1)	hydrology correctness is checked against known answers
Version control / CI	conventional commits, GitHub Actions (ruff, mypy strict, pytest, image build)	every push proves the build

version placed the top site on its bank (Figure 10); with the water body mask, 92.7 ha of water and its 200 m belt are excluded and the top site is a tributary (Figure 5): catchment 103 ha ±17 %, 75 % dependable rainfall 1 192 mm at the site (the overlay's 1 151 mm is the village-centroid series), CN 83, SCS-CN expected volume 172 126 m^3 ±30 % (rational 462 k, Strange 49 k: spread 182 %, reported), pond 50 000 m^3 (size cap binds), 126 × 126 m × 3.5 m, fills in 25 of 25 years, INR 97 lakh indicative, confidence *moderate* (*low* when SoilGrids is unreachable and the soil group is assumed).

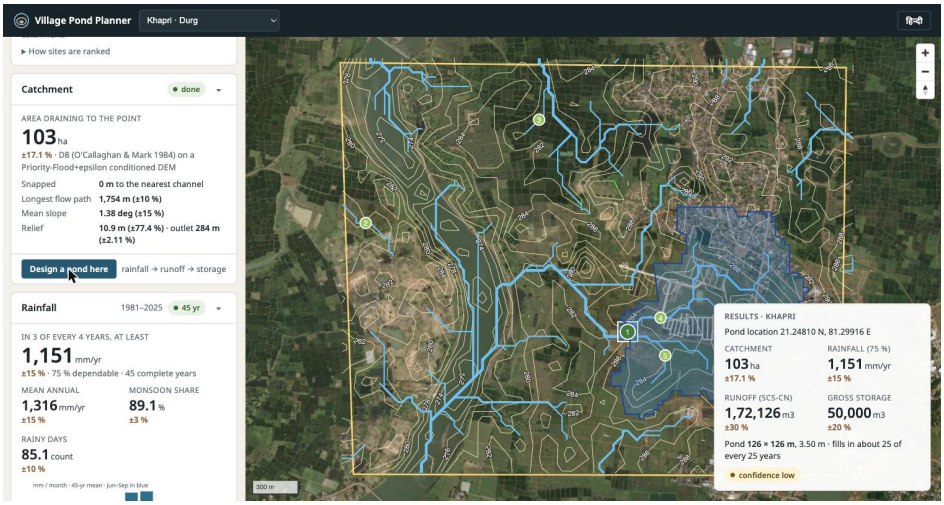


Fig. 5. Example results overlay for the provided contour map: ranked sites (all ≥ 295 m from mapped water), the top site's 103 ha catchment, and the pond result.

7.1 Testing and Validation

(i) 41 golden tests with analytic answers: inclined plane and cone interpolation (RMS <0.25 m), pit filling to spill level, plane accumulation equal to the upslope cell count, V-valley catchment equal to the rectangle upstream, Y-valley Strahler order 2, edge-fed flags on a hill and a tilted plane, river and water exclusion with the flood belt, SCS-CN and frustum hand calculations. (ii) **Independent implementation:** pysheds [2] on the same DEM agrees within 2.0 % and 3.4 % on the main outlets (22.5 % on a floodplain flat where flat-resolution schemes differ). (iii) **Independent elevation source:** GLO-30 against the contour DEM on the sample — surfaces agree (RMSE 1.8 m after a −5 m datum offset, r=0.94) but on this flat terrain (mean slope 1.6°) small flow paths do not, and catchments at small sites differ several-fold (Figure 11); the algorithm is not the uncertainty, the source DEM is — hence the bands. (iv) Snapping cuts the pour-point coefficient of variation from 212 % to 48 %. (v) The two OSM-mapped tanks in the sample are canal-fed; local runoff is a tenth of their capacity — recorded honestly as a mismatch of purpose, not of method.

7.2 Performance

Locust, 20 concurrent users (10 clicking catchments, 10 drawing a *new* box each time, so no cache helps), 90 s, one process (8-core laptop, live GLO-30 and Open-Meteo): `POST /analyzeArea` p95 17 ms; full area analysis p50 2 s, p95 4 s (154 in 90 s); catchment end to end p95 530 ms; rainfall p95 63 ms. Docker stack, 50 users: POST p95 33 ms, end-to-end p95 560 ms, no failures. The 46-check HTTP suite runs against the load-balanced lab URL. **Limits within the four systems:** one process sustained 1.7 area analyses per second; the 25 km^2 cap bounds one request to seconds; 429 backpressure caps each queue at 20 jobs; replicas share nothing, so throughput should grow roughly linearly with the four lab replicas behind the balancer (expected, not yet measured on the lab network).

8 DISCUSSION AND LIMITATIONS

What worked: one chain behind a DEM port served the upload and the map selection unchanged; every edge case found on the map (a pond on the river, a truncated catchment, a flattened river) became a hard rule with a golden test and a named warning. **Limits:** both DEMs are 30 m (SRTM ±6 m and GLO-30 ±2 m relative); relief below a few metres is not real, and on flat land the catchment of a small site is dominated by DEM error — the reason for the ±15–50 % bands and the advice to verify the outlet in the field. GLO-30 includes trees and buildings; reanalysis rainfall is about ±15 %; the runoff methods disagree by up to 2× (SCS-CN is the design figure, the range is shown); ownership is never assumed; the ML scorer waits for labels; lab replicas forget villages on restart. The lab VMs' DNS fails ~40 % of lookups, handled by a retrying, stale-if-error resolver cache and retried tile reads.

9 AI TOOL USAGE DECLARATION

An AI coding assistant (Claude Code, Anthropic) was used under the author's direction and review, as the assignment's LLM policy permits, for: refactoring and structuring code into the layered design; brainstorming and understanding the algorithms (D8 as a functional graph, why descending elevation is a topological order, catchment as reverse BFS, the river and truncation edge cases); implementing features to the author's specification with tests alongside (including the map-selection adapter and the lab deployment); debugging defects found by running the system; and editing this report and its figures, reformatting it into this template. Every number in the report is produced by the code. Every design decision is recorded with its reasoning and rejected alternatives in 21 architecture decision records and the decision log (`docs/PROGRESS.md`); the author has reviewed, understood and can explain each component.

REFERENCES

[1] Richard Barnes, Clarence Lehman, and David Mulla. 2014. Priority-Flood: An Optimal Depression-Filling and Watershed-Labeling Algorithm for Digital Elevation Models. *Computers & Geosciences* 62 (2014), 117–127. https://doi.org/10.1016/j.cageo.2013.04.024

[2] Matt Bartos. 2020. pysheds: Simple and Fast Watershed Delineation in Python. https://doi.org/10.5281/zenodo.3822494

[3] Keith J. Beven and Michael J. Kirkby. 1979. A Physically Based, Variable Contributing Area Model of Basin Hydrology. *Hydrological Sciences Bulletin* 24, 1 (1979), 43–69. https://doi.org/10.1080/02626667909491834

[4] David H. Douglas and Thomas K. Peucker. 1973. Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line or its Caricature. *Cartographica* 10, 2 (1973), 112–122. https://doi.org/10.3138/FM57-6770-U75U-7727

[5] European Space Agency and Airbus. 2022. Copernicus DEM GLO-30: Global Digital Elevation Model, Product Handbook. https://doi.org/10.5270/ESA-c5d3d65 Cloud-optimised tiles: AWS Open Data Registry, s3://copernicus-dem-30m.

[6] Emil J. Gumbel. 1958. *Statistics of Extremes.* Columbia University Press, New York.

[7] Berthold K. P. Horn. 1981. Hill Shading and the Reflectance Map. *Proc. IEEE* 69, 1 (1981), 14–47. https://doi.org/10.1109/PROC.1981.11918

[8] Stuart K. McFeeters. 1996. The Use of the Normalized Difference Water Index (NDWI) in the Delineation of Open Water Features. *International Journal of Remote Sensing* 17, 7 (1996), 1425–1432. https://doi.org/10.1080/01431169608948714

[9] Joaquín Muñoz-Sabater et al. 2021. ERA5-Land: a State-of-the-Art Global Reanalysis Dataset for Land Applications. *Earth System Science Data* 13 (2021), 4349–4383. https://doi.org/10.5194/essd-13-4349-2021

[10] John F. O'Callaghan and David M. Mark. 1984. The Extraction of Drainage Networks from Digital Elevation Data. *Computer Vision, Graphics, and Image Processing* 28, 3 (1984), 323–344. https://doi.org/10.1016/S0734-189X(84)80011-0

[11] Nobuyuki Otsu. 1979. A Threshold Selection Method from Gray-Level Histograms. *IEEE Transactions on Systems, Man, and Cybernetics* 9, 1 (1979), 62–66. https://doi.org/10.1109/TSMC.1979.4310076

[12] Laura Poggio et al. 2021. SoilGrids 2.0: Producing Soil Information for the Globe with Quantified Spatial Uncertainty. *SOIL* 7 (2021), 217–240. https://doi.org/10.5194/soil-7-217-2021

[13] Thomas L. Saaty. 1980. *The Analytic Hierarchy Process.* McGraw-Hill, New York.

[14] Arthur N. Strahler. 1957. Quantitative Analysis of Watershed Geomorphology. *Transactions, American Geophysical Union* 38, 6 (1957), 913–920. https://doi.org/10.1029/TR038i006p00913

[15] K. Subramanya. 2013. *Engineering Hydrology* (4th ed.). McGraw Hill Education, New Delhi.

[16] USDA Natural Resources Conservation Service. 1986. *Urban Hydrology for Small Watersheds.* Technical Release 55. USDA.

[17] Lei Wang and Hongxing Liu. 2006. An Efficient Method for Identifying and Filling Surface Depressions in Digital Elevation Models for Hydrologic Analysis and Modelling. *International Journal of Geographical Information Science* 20, 2 (2006), 193–213. https://doi.org/10.1080/13658810500433453

[18] Daniele Zanaga et al. 2022. ESA WorldCover 10 m 2021 v200. https://doi.org/10.5281/zenodo.7254221

[19] Lyle W. Zevenbergen and Colin R. Thorne. 1987. Quantitative Analysis of Land Surface Topography. *Earth Surface Processes and Landforms* 12, 1 (1987), 47–56. https://doi.org/10.1002/esp.3290120107

A SOURCE CODE AND REPOSITORY

Repository: https://github.com/Rahul5977/CSD-Course/tree/main/Project/1.PondPlanning (public). **Working front-end:** http://10.1.75.53:4272 (Swagger at `/docs`, planner at `/app`). **Demo video (5 min):** https://www.youtube.com/watch?v=MsWABr4THGA. **Run anywhere:** `make up && make seed` (Docker, 11 services) or `make serve-single` (one process); `make check` runs lint, types and tests. **Structure:** `app/` (`api` routers, `schemas`, `engines` — terrain, hydrology, rainfall, runoff, design, suitability, workflows —, `domain`, `providers`, `repositories`, `jobs`), `web/` (React SPA), `tests/` (incl. `golden/`), `infra/` (Compose, Dockerfiles, nginx, lab replica script and load balancer), `docs/` (ADRs, API cookbook and error catalogue, figures, progress log), `scripts/` (figures, cross-checks, e2e suite).

B ADDITIONAL FIGURES

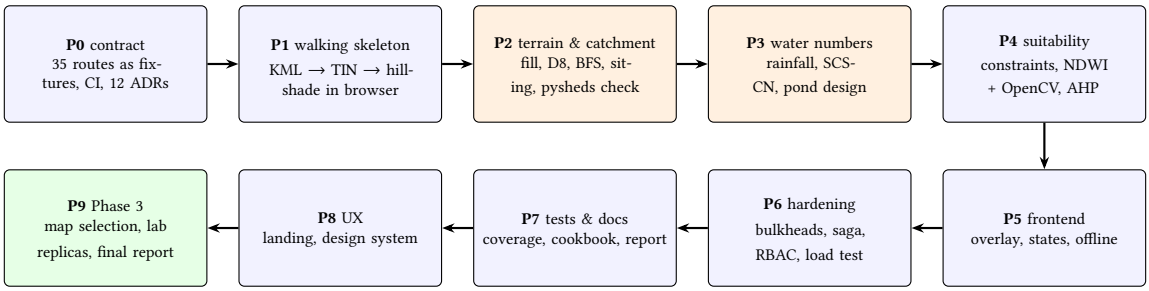


Fig. 6. Work flow of the project: phases P0–P9, each closed by a gate with evidence (test output, figure or recording) before the next began. Orange: the examined core.

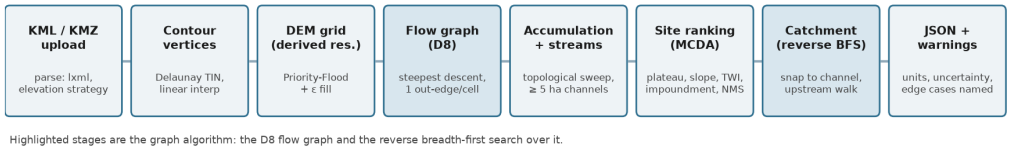


Fig. 7. The analysis pipeline; the highlighted stages form the graph algorithm.

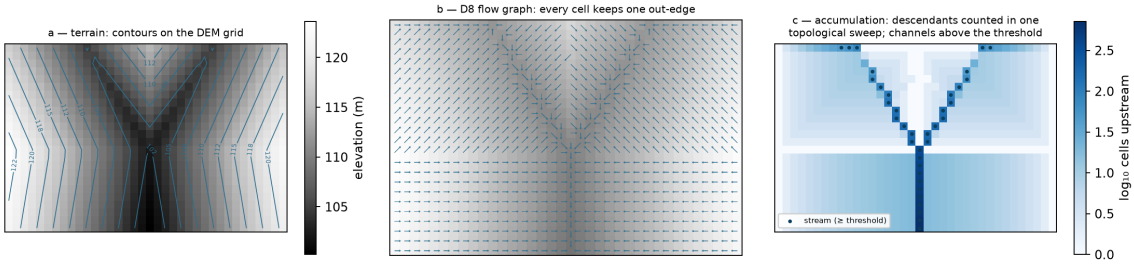


Fig. 8. DEM → D8 flow graph → accumulation and streams (generated from the engine code).

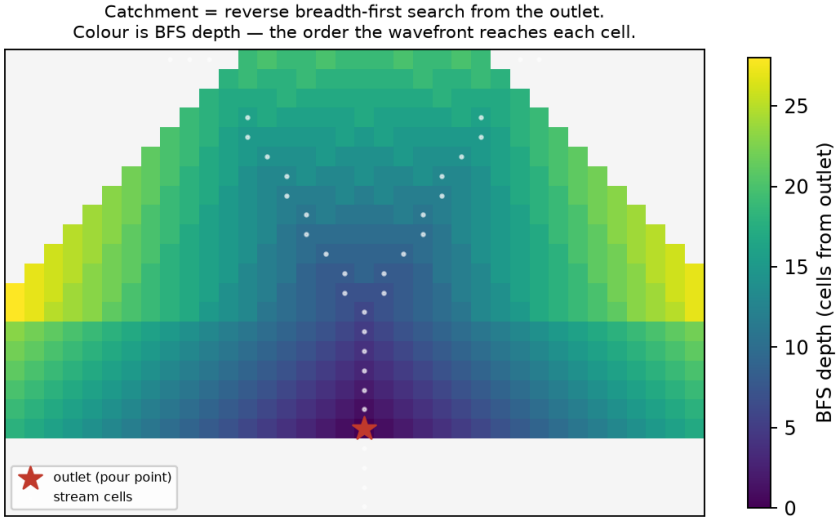


Fig. 9. Catchment as a reverse BFS from the outlet; colour is the wavefront order.

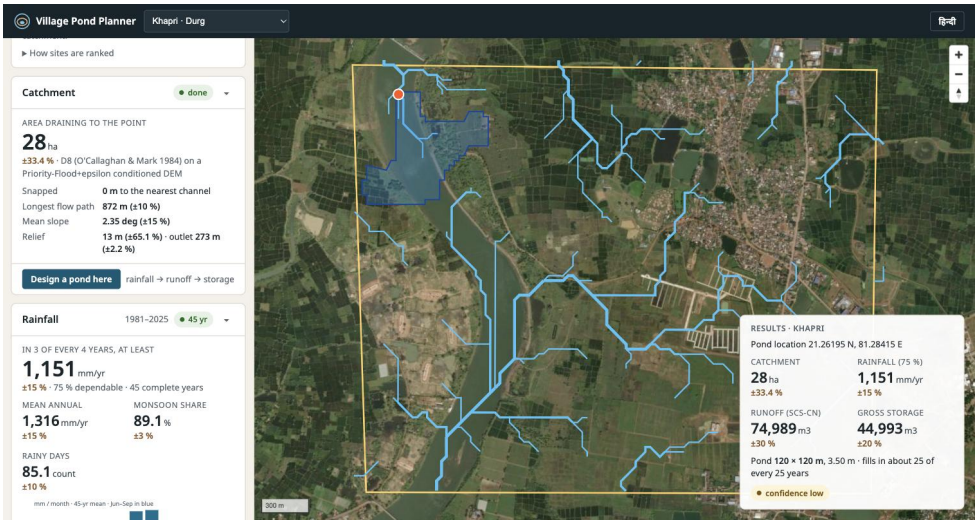


Fig. 10. Before the water body mask: the top site on the Shivnath's bank, its catchment 40 % permanent water. Compare Figure 5.

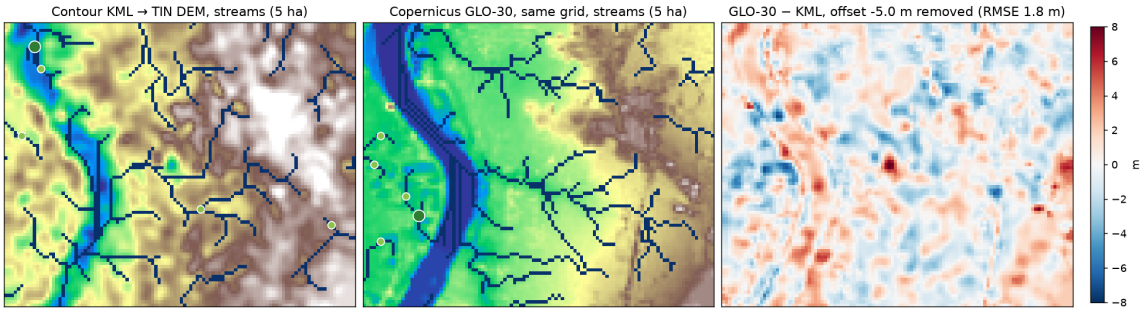


Fig. 11. Terrain-source cross-check on the sample: contour DEM vs. GLO-30 on the same grid, streams and sites from the identical chain, and the elevation difference.

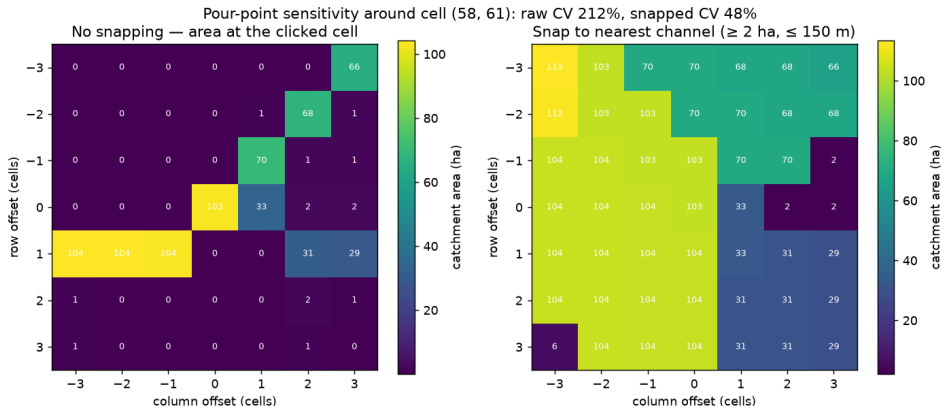


Fig. 12. Pour-point sensitivity: catchment area around a channel cell, raw vs. snapped.

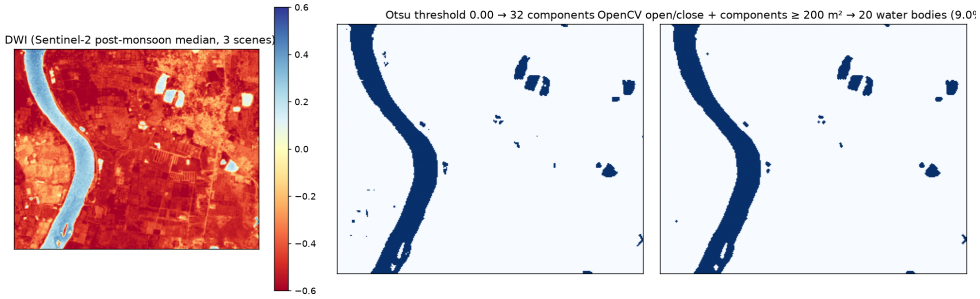


Fig. 13. NDWI water mask with OpenCV: Otsu threshold, morphology, connected components.